# Module Resolution

10 min read · Updated 24 Jul 2026 · Topic 13 of 70



## Why it matters

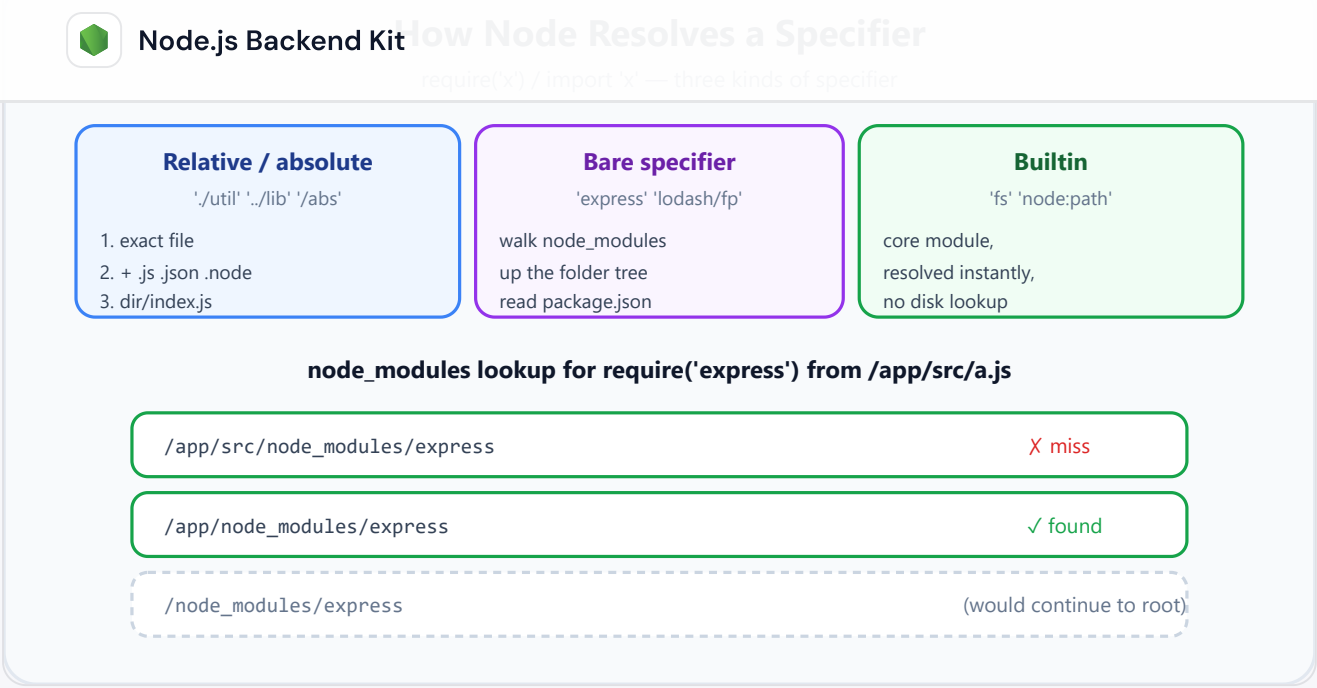
Module resolution is the algorithm that turns a short string like `'express'` or `'./utils'` into one exact file on disk. It runs on every `require` and every `import`, and when it goes wrong you get the two most common Node errors in existence: `MODULE_NOT_FOUND` and `ERR_MODULE_NOT_FOUND`. Understanding resolution is what lets you answer questions that otherwise feel like black magic: "Why did it find version 2 of this package and not version 3?", "Why does `import 'pkg/internal'` suddenly fail after an upgrade?", "Why does my monorepo pick the wrong copy of a dependency?" These are all resolution questions.

Resolution is also central to modern package design. The `exports` field and conditional exports let a package author decide precisely which files consumers can reach and whether they get the ESM or CommonJS build. If you publish libraries — or debug why a dependency behaves differently under `import` than `require` — resolution is the topic that explains it.

## The concept, simply

Every specifier you pass falls into one of three buckets, and each is resolved differently:

- **Relative or absolute** — starts with `./`, `../`, or `/`. Node treats it as a path from the current file (or filesystem root) and looks for a matching file or directory.
- **Bare specifier (a package name)** — `'express'`, `'lodash/fp'`, `'@scope/pkg'`. Node searches `node_modules` folders, walking up the directory tree.
- **Builtin** — `'fs'`, `'path'`, or the explicit `'node:fs'` form. Resolved instantly from Node itself, no disk lookup.




A specifier is relative/absolute, a bare package name, or a builtin. Package names trigger a walk up successive node_modules folders until the package is found or the filesystem root is reached.

## How it works

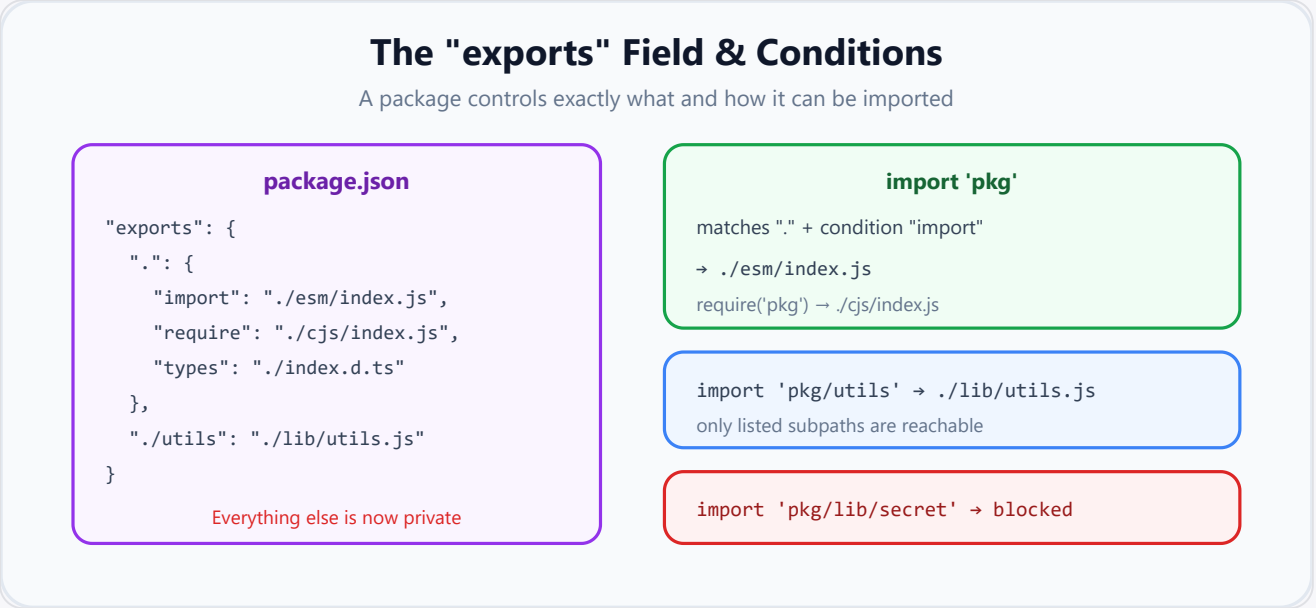
**Relative/absolute resolution (CommonJS).** Given `require('./util')`, Node tries, in order: (1) the exact path as a file — `./util`; (2) the path plus extensions — `./util.js`, then `./util.json`, then `./util.node`; (3) the path as a directory — it reads `./util/package.json` and uses its `main` field, or falls back to `./util/index.js`. The first hit wins. This "try several extensions and an index file" behaviour is the convenient extension guessing that CommonJS is known for.

**Relative resolution (ESM) is stricter.** ES Modules do *not* guess extensions and do *not* auto-resolve `index.js` for a bare directory. `import './util'` fails; you must write `import './util.js'` or `import './util/index.js'` explicitly. This strictness is intentional — it makes resolution unambiguous and matches browser behaviour.

**Bare specifier resolution — the `node_modules` walk.** For `require('express')` from `/app/src/server.js`, Node looks for `express` in a series of `node_modules` directories, starting next to the file and climbing toward the root: `/app/src/node_modules/express`, then `/app/node_modules/express`, then `/node_modules/express`. The first one found wins. This upward walk is exactly why a nested dependency can have its own private copy of a package (in its local

**Inside a found package.** Once Node locates `node_modules/express`, it reads that package's `package.json` to decide the entry file. Modern resolution consults the `exports` field first (if present); otherwise it uses `main` (and, for some tools, `module` or `browser`); with neither, it falls back to `index.js`. A subpath like `'lodash/fp'` means "the `fp` entry inside the `lodash` package," resolved either through the package's `exports` subpaths or as a literal file path when no `exports` map exists.

**The `exports` field — encapsulation.** When a package declares `"exports"`, it becomes the *only* gateway into the package. Any file not listed is unreachable, even if it physically exists — `import 'pkg/lib/internal.js'` throws `ERR_PACKAGE_PATH_NOT_EXPORTED`. This lets authors ship internal files without them becoming part of the public API. The map keys are subpaths (`"."` is the package root, `"./utils"` a named subpath) and the values are the real files.




The exports field maps public subpaths to real files and can branch on conditions like import vs require; anything not listed becomes private and un-importable.

**Conditional exports.** An entry can be an object whose keys are *conditions* Node evaluates against the current environment, matched top-to-bottom (order matters):

- `"import"` — used when the package is loaded via `import` / `await import()` (ESM).
- `"require"` — used when loaded via `require()` (CJS).

- `"types"` — where TypeScript should look for declarations (must come first among conditions).

This is how one package serves an ESM build to `import` and a CommonJS build to `require` from the same install — the classic "dual package." It is also the mechanism behind subtle bugs where the `import` and `require` paths load different files with slightly different behaviour.

**Conditions are matched, not merged.** Node walks the condition keys of an `exports` object *in the order they are written* and takes the first one that applies to the current request, then stops — it never blends two branches. That single rule explains most misconfigured packages: if `"default"` sits above `"import"`, the ESM branch is dead code because `"default"` already matched; if `"types"` is not first, TypeScript resolves a JavaScript file before it ever sees the declarations. A request also carries more than one active condition at once — an `import` from Node matches *both* `"import"` and `"node"` — so specificity, not just presence, decides the winner, and the author controls that purely through key order. The table below shows what is active for common load styles.

| How the package is loaded | Active conditions (most to least specific) |
|---|---|
| `import 'pkg'` in Node | `node`, `import`, `module-sync`, `default` |
| `require('pkg')` in Node | `node`, `require`, `default` |
| bundled for the browser | `browser`, `import` / `require`, `default` |
| type-checked by TypeScript | `types` (must precede the runtime keys), then the runtime set |

**Self-referencing and the resolution's endpoint.** A useful consequence of the `exports` map is that a package can import *itself* by name — `import { thing } from 'mylib'` from inside `mylib` resolves through its own `exports`, so tests and internal files exercise the exact public surface consumers see, not a relative shortcut that skips the

and that fully-resolved string — not the original specifier — is the key used for caching. Two different specifiers that resolve to the same real file therefore share one cached instance, which is why a symlinked or path-aliased dependency can accidentally load twice if the two routes normalise to different absolute paths.

**Other resolution features.** `imports` (with keys starting `#`) defines private, internal-only aliases usable within the package (`import x from '#internal/thing'`). The `NODE_PATH` environment variable adds extra global lookup directories but is discouraged. `require.resolve('pkg')` runs the algorithm and returns the resolved path without loading — invaluable for debugging "which file is this actually?".

## Code examples

**1. Inspect what a specifier resolves to.**

JAVASCRIPT

Copy

```javascript
// Prints the exact file Node would load — great for debugging duplicates.
console.log(require.resolve('express'));
// e.g. /app/node_modules/express/lib/express.js
console.log(require.resolve('./utils'));
// e.g. /app/src/utils/index.js
console.log(module.paths); // the ordered node_modules dirs Node will sear
```

**2. A dual-package `exports` map.**

JSON

Copy

```json
{
  "name": "mylib",
  "version": "1.0.0",
  "type": "module",
  "exports": {
    ".": {
      "types": "./dist/index.d.ts",
      "import": "./dist/index.mjs",
```

```
    "./utils": "./dist/utils.js"
  }
}
```

### 3. Encapsulation in action.

JAVASCRIPT Copy

```
import mylib from 'mylib';          // → dist/index.mjs (import condition
import u from 'mylib/utils';        // → dist/utils.js (listed subpath)
import x from 'mylib/dist/secret.js';
// throws ERR_PACKAGE_PATH_NOT_EXPORTED — not listed in "exports"
```

### 4. Internal imports with the `#` prefix.

JSON Copy

```
{
  "imports": {
    "#config": "./src/config.js",
    "#utils/*": "./src/utils/*.js"
  }
}
```

JAVASCRIPT Copy

```
// Anywhere inside the package:
import config from '#config';          // → ./src/config.js
import { slug } from '#utils/string'; // → ./src/utils/string.js
```

## Common mistakes & gotchas

- **Relying on extension guessing in ESM.** Works in CJS, fails in ESM — always write the full filename in `import`.

- **Reaching into a package's internals.** `import 'pkg/lib/x'` breaks when the author adds an `exports` field. Use only documented entry points.
- **Duplicate copies of a package.** The upward `node_modules` walk can load two versions of the same library (e.g. two React copies), causing "invalid hook" style bugs. Diagnose with `require.resolve` and `npm ls pkg`.
- **Wrong condition order.** Putting `"default"` before `"import"` in `exports` makes `"default"` always win; conditions match top-down, so specific ones come first and `"default"` last.
- **Forgetting `"types"` first.** TypeScript needs the `"types"` condition to appear before `"import"` / `"require"` or it won't find declarations.
- **Assuming `main` still applies with `exports`.** When `exports` is present it takes precedence for module resolution; `main` becomes a fallback for legacy tools only.
- **Casing on case-insensitive filesystems.** `require('./Util')` may work on macOS/Windows but fail on Linux/CI. Match the real filename exactly.
- **Abusing `NODE_PATH`.** It hides real dependencies and is non-portable; declare deps in `package.json` instead.

## Best practices

- Use `require.resolve('pkg')` and `npm ls pkg` to confirm exactly which file/version is loaded when debugging.
- Import packages only through their documented entry points, never deep internal paths.
- When publishing, define an `exports` field to encapsulate internals and to serve correct ESM/CJS builds via conditions.
- Order conditions carefully: `"types"` first, then `"import"` / `"require"`, with `"default"` last.
- Prefer the `#`-prefixed `imports` map for internal aliases over brittle relative-path chains.
- Write full file extensions in every relative ESM specifier.
- Keep the dependency tree flat and deduped (lockfile + `npm dedupe`) to avoid duplicate-copy bugs.

**Q: Walk me through how Node resolves `require('express')`.** It is a bare specifier, so Node searches `node_modules` folders from the requiring file's directory upward: `./node_modules/express`, `../node_modules/express`, and so on to the filesystem root. On the first hit it reads that package's `package.json`, consulting `exports` (else `main`, else `index.js`) to pick the entry file.

**Q: How does resolving `'./util'` differ between CJS and ESM?** CJS tries `util`, then `util.js/.json/.node`, then `util/index.js` — it guesses extensions and index files. ESM does none of that; you must write the exact filename with extension. Trap: code that "worked" is really relying on CJS guessing and breaks on the switch to ESM.

**Q: What does the `exports` field do that `main` doesn't?** It *encapsulates* the package — only listed subpaths are importable, everything else becomes private (`ERR_PACKAGE_PATH_NOT_EXPORTED`) — and it supports *conditional* resolution (different files for `import` vs `require`, node vs browser). `main` is a single legacy entry point with no encapsulation or conditions.

**Q: How can one `require` and `import` of the same package load different code?** Via conditional exports: `"exports": { ".": { "import": "./esm.js", "require": "./cjs.js" } }`. Node picks the branch matching how the package was loaded. This is the dual-package pattern (and a source of "dual package hazard" if the two builds hold separate state).

**Q: You suspect two versions of a library are loaded. How do you confirm?** `require.resolve('lib')` from different files shows the actual paths; `npm ls`

`tree` shows every version in the tree. Duplicates arise because the `node_modules` walk resolves from each file's location, so nested deps can pin their own copy.

**Q: What is the `#` in `import '#config'`?** A package-internal import defined in the `imports` field of `package.json`. It creates a private alias usable only inside that package — a clean alternative to long `../../` relative chains.

## Cheat-card

- **Three specifier types:** relative/absolute · bare (package) · builtin (`node:`).
- **CJS relative:** exact → +`.js/.json/.node` → `dir/index.js`. **ESM:** exact filename only.
- **Bare:** walk `node_modules` upward to root; first hit wins.
- **Entry pick:** `exports` → `main` → `index.js`.
- `exports` **field** encapsulates: unlisted paths = `ERR_PACKAGE_PATH_NOT_EXPORTED`.
- **Conditions** (top-down): `types` → `import` / `require` / `node` / `browser` → `default` last.
- `imports` + `#` = private internal aliases.
- **Debug:** `require.resolve('x')`, `module.paths`, `npm ls x`.

QUICK CHECK — no marks, just for you.

1. How does Node resolve a bare specifier like `require('express')`?

- ○ It downloads express from the npm registry on demand
- ○ It looks only in the global npm folder
- ○ It walks up `node_modules` directories from the requiring file toward the filesystem root; first hit wins
- ○ It reads the `NODE_PATH` variable exclusively

2. What does adding an `exports` field to a package do that `main` does not?

- ◯ Nothing — they are equivalent
- ◯ It disables conditional resolution
- ◯ It makes `main` mandatory
- ◯ It encapsulates the package so only listed subpaths are importable, and it supports conditional resolution

Check answer

3. How does resolving `./util` differ between CommonJS and ESM?

- ◯ CJS guesses extensions and index files; ESM requires the exact filename with extension
- ◯ Both require the exact filename
- ◯ ESM guesses extensions while CJS requires the full path
- ◯ They resolve identically

Check answer

Node.js Backend Kit

LEARN MORE

## ARTICLES & DOCS

Modules: Packages (resolution, exports, imports)
nodejs.org

Modules: CommonJS modules (module resolution algorithm)
nodejs.org

## VIDEOS




Node JS Tutorial for Beginners #7 - Module Patterns
Net Ninja

Hand-picked references for this lesson.